

PROCESS

Contents_ai_api

PDF → Layout JSON / Full TOC JSON 생성 API

Contents_ai_api는 PDF 파일을 입력으로 받아 아래 순서로 목차 추출용 JSON 파일을 자동 생성합니다.

PDF 업로드 -> PDF layout text 파싱 -> OpenAI layout rule 생성 -> Gemma/vLLM 최종 TOC 생성 -> 결과 저장

처리는 비동기 job 방식입니다. API 요청을 보내면 결과를 바로 기다리지 않고 `job_id` 를 먼저 받고, 이후 상태 조회 API로 완료 여부와 결과 파일을 확인합니다.

필요 설치

```
pip install -r requirements.txt
```

필수 패키지:

- `fastapi`
- `uvicorn`
- `python-multipart`
- `pdfplumber`

1. 실행 환경 설정

1.1 프로젝트 위치

```
cd /path/to/Contents_ai_api
```

예시:

```
cd ~/workspace/Contents_ai_test/Contents_ai_api
```

1.2 가상환경

새로 만들 때:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

이미 가상환경이 있으면 활성화만 합니다.

```
source .venv/bin/activate
```

1.3 OpenAI API Key

OpenAI layout 생성에는 프로젝트 루트의 `.env` 파일에 아래 값이 있으면 됩니다.

```
OPENAI_API_KEY=sk-...
```

`.env` 를 새로 만들거나 수정한 뒤에는 FastAPI 서버를 재시작해야 합니다.

1.4 Gemma/vLLM 서버

최종 TOC 생성에는 Gemma/vLLM OpenAI-compatible 서버가 필요합니다.

`/jobs/layout/upload` 만 사용할 때는 필요 없습니다.

```
CUDA_VISIBLE_DEVICES=4,5,6,7 VLLM_USE_FLASHINFER_SAMPLER=0 \
python -m vllm.entrypoints.openai.api_server \
  --model google/gemma-4-31B-it \
  --served-model-name google/gemma-4-31B-it \
  --host 127.0.0.1 \
  --port 8000 \
  --tensor-parallel-size 4 \
  --max-model-len 262144 \
  --gpu-memory-utilization 0.9 \
  --trust-remote-code
```

기본값:

- Gemma base URL: `http://127.0.0.1:8000/v1`
- Gemma model: `google/gemma-4-31B-it`

- Gemma API key: `EMPTY`

2. FastAPI 서버 실행

2.1 로컬 테스트용 실행

```
python full_toc_v4.py --host 127.0.0.1 --port 8080
```

2.2 외부 접속용 실행

```
python full_toc_v4.py --host 0.0.0.0 --port 8080
```

외부 PC에서 접속하려면 서버/보안망/방화벽에서 `8080` 포트가 열려 있어야 합니다.

2.3 서버 상태 확인

```
curl <http://127.0.0.1:8080/health>
```

정상 응답:

```
{"ok":true,"service":"full_toc_v4","framework":"fastapi"}
```

2.4 API 문서

```
<http://127.0.0.1:8080/docs>
```

3. 주요 파일 역할

3.1 `app.py`

- FastAPI 앱 생성
- `routes.py` 의 router 등록
- 서버 실행 인자 파싱

주요 함수:

- `create_app()`
- `parse_server_args()`
- `main()`

3.2 routes.py

- HTTP API 엔드포인트 정의
- multipart 업로드 파싱
- JSON payload 파싱
- job 생성 호출
- result/log/status 파일 다운로드 처리

주요 구성:

- `FORM_UPLOAD_FIELDS`
 - 허용 upload field: `file`, `files`, `pdf`, `pdfs`
- `parse_config()`
 - multipart form의 `config` 문자열을 JSON object로 변환
- `http_error()`
 - 에러 응답 형식을 통일
- `uploaded_paths_from_form()`
 - 업로드 PDF를 `data/uploads/`에 저장
- `stage_upload_endpoint()`
 - `/jobs/layout/upload`, `/jobs/gemma/upload`, `/jobs/toc/upload` endpoint 생성
- `stage_json_endpoint()`
 - `/jobs/layout`, `/jobs/gemma`, `/jobs/toc` endpoint 생성
- `job_file_path()`
 - result/log/status 다운로드 파일 경로 선택

3.3 jobs.py

- job 생성
- status 파일 생성

- background thread 실행
- pipeline 완료 후 상태 갱신

주요 함수:

- `start_background_job()`
 - job_id 생성
 - `status.json` 초기 상태 저장
 - background thread 시작
- `run_job()`
 - status를 `running` 으로 변경
 - `full_toc_core.run_pipeline_locked()` 실행
 - 결과에 따라 `completed` 또는 `failed` 저장
- `include_result_if_requested()`
 - `include_result=true` 일 때 결과 JSON 내용을 응답에 포함

3.4 storage.py

- job 상태 파일 저장/조회
- 업로드 PDF 저장
- 안전한 job_id 검증
- payload 요약 시 API key 숨김 처리

주요 함수:

- `make_job_id()`
- `write_status()`
- `read_status()`
- `save_uploaded_pdf_bytes()`
- `summarize_payload()`

3.5 full_toc_core.py

실제 파이프라인의 핵심 파일입니다.

- PDF layout text 추출

- headings 후보 필터링
- OpenAI layout JSON 생성
- Gemma/vLLM 최종 TOC 생성
- 결과 JSON 저장

주요 함수:

- `parse_pdf_if_needed()`
- `extract_region_lines()`
- `should_keep_formatted_line()`
- `run_layout_stage()`
- `run_gemma_stage()`
- `run_pipeline()`
- `run_pipeline_locked()`

3.6 full_toc_v4.py

- 서버 실행 wrapper
- `app.py` 의 `main()` 호출

3.7 smoke_test.py

- 외부 OpenAI/Gemma 서버 없이 API/job 흐름 검증
- 실제 모델 호출은 fake 응답으로 대체

4. API Endpoint

이 장은 endpoint 목록만 요약합니다. 각 endpoint의 상세 처리 순서는 6~8장을 참고합니다.

4.1 생성 API

POST /jobs/layout/upload	PDF 업로드 -> layout JSON만 생성
POST /jobs/gemma/upload	PDF 업로드 + 기존 layout JSON -> 최종 TOC JSON 생성
POST /jobs/toc/upload	PDF 업로드 -> layout JSON + 최종 TOC JSON 생성

4.2 JSON Body API

서버에 이미 있는 PDF 경로 또는 base64 입력을 JSON body로 넘길 때 사용합니다.

```
POST /jobs/layout
POST /jobs/gemma
POST /jobs/toc
```

4.3 Job 조회 / 파일 다운로드 API

GET /jobs/{job_id}	job 상태 전체 조회
GET /jobs/{job_id}/result	job 결과 메타데이터 조회
GET /jobs/{job_id}/files/result	생성된 결과 JSON 다운로드
GET /jobs/{job_id}/files/log	실행 로그 다운로드
GET /jobs/{job_id}/files/status	job status.json 다운로드

5. CLI / API config 옵션

API `config` JSON에서는 옵션명을 snake_case로 씁니다.

CLI에서는 같은 옵션을 kebab-case로 씁니다.

예:

- API: `parse_mode`
- CLI: `-parse-mode`

5.1 입력 옵션

- `input_file` / `-input-file`, `f`
 - 처리할 PDF 파일 경로입니다.
 - 서버에 이미 있는 파일을 JSON body 방식으로 실행할 때 사용합니다.
 - CLI에서는 여러 번 지정할 수 있습니다.
 - 예: `-input-file data/input/file.pdf`
- `input_dir` / `-input-dir`
 - `input_file` 을 지정하지 않았을 때 PDF를 찾을 기본 폴더입니다.
 - 폴더 아래의 `.pdf` 파일을 재귀적으로 찾습니다.
 - 기본값: `data/input`
- `file`

- multipart upload에서 사용하는 단일 PDF 필드입니다.
- 일반적인 업로드 API에서는 이 필드를 사용합니다.
- 예: `F 'file=@sample.pdf'`
- `files`
 - multipart 또는 JSON base64 방식의 다중 파일 입력입니다.
 - 여러 PDF를 한 번에 넘길 때 사용합니다.
- `filename`
 - JSON base64 업로드에서 저장 파일명을 지정할 때 사용합니다.
 - multipart 업로드에서는 업로드 파일명이 자동으로 사용됩니다.
- `file_base64`
 - JSON body로 PDF를 base64 문자열로 전달할 때 사용합니다.
 - 일반 curl 업로드에서는 거의 사용하지 않습니다.

5.2 stage 옵션

- `stage=layout` / `-stage layout`
 - PDF 파싱 후 OpenAI layout JSON까지만 생성합니다.
 - 생성 위치: `data/layout/`
 - Gemma/vLLM 서버는 필요 없습니다.
- `stage=gemma` / `-stage gemma`
 - 기존 layout JSON을 사용해 최종 TOC JSON만 생성합니다.
 - `layout_file` 이 필요하거나, `layout_dir` 에서 최신 layout을 자동 검색합니다.
 - Gemma/vLLM 서버가 필요합니다.
- `stage=all` / `-stage all`
 - layout 생성과 최종 TOC 생성을 모두 수행합니다.
 - OpenAI API key와 Gemma/vLLM 서버가 모두 필요합니다.
 - 기본값입니다.

API endpoint에 따라 stage는 자동 지정됩니다.


```
/jobs/layout/upload -> stage="layout"  
/jobs/gemma/upload -> stage="gemma"  
/jobs/toc/upload -> stage="all"
```

5.3 저장 경로 옵션

- `data_dir` / `-data-dir`
 - 전체 산출물의 기준 폴더입니다.
 - 기본값: `data`
 - 하위 폴더가 기본값이면 `parsed`, `layout`, `full_json`, `log` 경로도 이 폴더 아래로 맞춰집니다.
- `parsed_dir` / `-parsed-dir`
 - PDF parsed text 캐시 저장 폴더입니다.
 - 기본 위치: `data/parsed`
- `layout_dir` / `-layout-dir`
 - OpenAI layout JSON 저장/검색 폴더입니다.
 - 기본 위치: `data/layout`
- `full_json_dir` / `-full-json-dir`
 - Gemma/vLLM 최종 TOC JSON 저장 폴더입니다.
 - 기본 위치: `data/full_json`
- `log_dir` / `-log-dir`
 - 실행 로그 저장 폴더입니다.
 - 기본 위치: `data/log`
- `layout_file` / `-layout-file`
 - `stage=gemma` 에서 사용할 layout JSON 경로입니다.
 - 지정하지 않으면 `layout_dir` 에서 해당 PDF의 최신 layout 파일을 찾습니다.
 - 여러 PDF를 처리할 때는 사용할 수 없고, 단일 PDF에만 사용합니다.

5.4 PDF 파싱 옵션

- `parse_mode` / `-parse-mode`

- PDF parsed text 저장 방식입니다.
- `full`
 - PDF 본문 전체 layout text를 저장합니다.
 - 모델 입력이 커질 수 있습니다.
- `headings`
 - heading 후보 중심으로 줄인 parsed text를 저장합니다.
 - TOC 생성 API에서는 보통 `headings` 를 권장합니다.
- `force_parse` / `-force-parse`
 - 기존 parsed text가 있어도 PDF를 다시 파싱합니다.
 - 필터 로직을 수정한 뒤 기존 캐시를 무시하고 재생성할 때 유용합니다.
- `pdf_column_mode` / `-pdf-column-mode`
 - PDF 컬럼 분리 방식입니다.
 - `auto`
 - 페이지별로 2단 여부를 자동 감지합니다.
 - `none`
 - 컬럼 분리를 하지 않습니다.
 - `two`
 - 항상 2단으로 처리합니다.
 - 기본값: `auto`
- `pdf_column_split_x` / `-pdf-column-split-x`
 - 2단 컬럼을 나눌 x 좌표입니다.
 - `0` 이면 페이지 중앙을 사용합니다.
- `pdf_column_gap` / `-pdf-column-gap`
 - 컬럼 분리선 주변에서 제외할 gutter 폭입니다.
 - 기본값: `8.0`
- `max_heading_chars` / `-max-heading-chars`
 - `parse_mode=headings` 에서 heading 후보로 남길 최대 글자 수입니다.
 - 이 길이를 넘는 라인은 본문으로 보고 제거합니다.

- 기본값: `140`
- `drop_author_lines` / `-drop-author-lines`, `-no-drop-author-lines`
 - 저자명처럼 보이는 라인을 제거할지 결정합니다.
 - 기본값: `true`
- `drop_body_size_lines` / `-drop-body-size-lines`, `-no-drop-body-size-lines`
 - 본문 기본 크기와 같은 라인을 제거할지 결정합니다.
 - `headings` 모드에서 본문이 heading rule로 섞이는 것을 줄입니다.
 - 기본값: `true`
- `body_size_ratio_tolerance` / `-body-size-ratio-tolerance`
 - 본문 크기와 같다고 볼 ratio 허용 범위입니다.
 - 기본값: `0.08`
- `gemma_include_full_context` / `-gemma-include-full-context`, `-no-gemma-include-full-context`
 - `parse_mode=headings` 일 때 Gemma prompt에 전체 parsed 본문을 문맥으로 함께 넣을지 결정합니다.
 - 기본값: `true`

5.5 TOC 출력 구조 옵션

- `title` / `-title`
 - 결과 TOC의 문서 제목입니다.
 - 지정하지 않으면 PDF 파일명 stem을 사용합니다.
- `prompt` / `-prompt`
 - Gemma/vLLM에 전달할 사용자 지시문입니다.
 - 기본값은 전체 문서의 완전한 TOC JSON 생성을 요청하는 문장입니다.
- `max_depth` / `-max-depth`
 - TOC level 최대 깊이입니다.
 - 기본값: `7`

5.6 OpenAI layout 생성 옵션

- `openai_api_key` / `-openai-api-key`

- OpenAI API key입니다.
- 보통 요청에 직접 넣지 않고 `.env` 의 `OPENAI_API_KEY` 를 사용합니다.
- `openai_model` / `-openai-model`
 - layout JSON 생성에 사용할 OpenAI 모델입니다.
 - 환경변수 `OPENAI_LAYOUT_MODEL` 또는 `OPENAI_MODEL` 로도 지정할 수 있습니다.
- `openai_base_url` / `-openai-base-url`
 - OpenAI-compatible endpoint입니다.
 - 기본값: `https://api.openai.com/v1`
- `openai_timeout` / `-openai-timeout`
 - OpenAI 요청 timeout 초 단위입니다.
- `openai_max_tokens` / `-openai-max-tokens`
 - layout 생성 응답의 최대 token 수입니다.
- `openai_temperature` / `-openai-temperature`
 - OpenAI layout 생성 temperature입니다.
 - 기본값은 지정하지 않습니다.
- `openai_retries` / `-openai-retries`
 - OpenAI 요청 실패 시 retry 횟수입니다.
- `openai_retry_base_delay` / `-openai-retry-base-delay`
 - OpenAI retry 대기 시간 계산의 기본 delay입니다.

5.7 Gemma/vLLM TOC 생성 옵션

- `model` / `-model`
 - Gemma/vLLM served model name입니다.
 - 기본값: `google/gemma-4-31B-it`
 - `31b` 처럼 축약하면 내부에서 `gemma4:31b` 형태로 정규화할 수 있습니다.
- `ai_fallback_models` / `-ai-fallback-models`
 - 기본 모델 실패 시 순서대로 시도할 fallback 모델 목록입니다.
 - 쉼표로 구분합니다.

- `base_url` / `-base-url`
 - Gemma/vLLM OpenAI-compatible endpoint입니다.
 - 기본값: `http://127.0.0.1:8000/v1`
- `api_key` / `-api-key`
 - Gemma/vLLM endpoint bearer key입니다.
 - 기본값: `EMPTY`
- `timeout` / `-timeout`
 - Gemma/vLLM 요청 timeout 초 단위입니다.
- `temperature` / `-temperature`
 - Gemma/vLLM 생성 temperature입니다.
 - 기본값: `0.0`
- `max_output_tokens` / `-max-output-tokens`
 - Gemma/vLLM 출력 token 제한입니다.
 - 출력이 잘리면 최종 JSON 파싱이 실패할 수 있습니다.
- `ai_retries` / `-ai-retries`
 - Gemma/vLLM 요청 또는 JSON 파싱 실패 시 retry 횟수입니다.
- `ai_retry_base_delay` / `-ai-retry-base-delay`
 - Gemma/vLLM retry 대기 시간 계산의 기본 delay입니다.

5.8 Gemma chunk 옵션

- `gemma_chunk_mode` / `-gemma-chunk-mode`
 - Gemma TOC 생성 시 문서를 나누는 방식입니다.
 - `tokens`
 - token 수 기준으로 chunk를 나눕니다.
 - `pages`
 - page 수 기준으로 chunk를 나눕니다.
 - 기본값: `tokens`
- `gemma_max_context_tokens` / `-gemma-max-context-tokens`

- vLLM 서버의 `-max-model-len` 값과 맞추는 context 한도입니다.
- 기본값: `262144`
- `gemma_chunk_token_limit` / `-gemma-chunk-token-limit`
 - chunk별 parsed text 목표 token 수입니다.
 - `0` 이면 context 한도와 safety token 기준으로 자동 계산합니다.
- `gemma_chunk_safety_tokens` / `-gemma-chunk-safety-tokens`
 - chunk 자동 계산 시 남겨둘 여유 token 수입니다.
- `gemma_chunk_pages` / `-gemma-chunk-pages`
 - `gemma_chunk_mode=pages` 일 때 chunk당 target page 수입니다.
 - 기본값: `80`
- `gemma_chunk_overlap_pages` / `-gemma-chunk-overlap-pages`
 - chunk 경계 문맥용 overlap page 수입니다.
 - 기본값: `2`

5.9 디버그/응답 옵션

- `write_raw` / `-write-raw`
 - OpenAI/Gemma raw 응답을 파일로 함께 저장합니다.
 - 모델 응답 디버깅이 필요할 때 사용합니다.
- `include_result`
 - API 전용 옵션입니다.
 - `true` 면 `/jobs/{job_id}/result` 응답에 생성된 JSON 파일 내용도 포함합니다.
 - 큰 결과에서는 응답이 커질 수 있습니다.

6. Layout 생성 프로세스

6.1 요청

```
curl -X POST <http://127.0.0.1:8080/jobs/layout/upload> \
-F 'file=@./Contents_ai_test/data/input/1_2.pdf' \
-F 'config={"parse_mode":"headings}"'
```

6.2 응답

```
{  
  "job_id": "20260727_110700_f071ed37",  
  "status": "queued",  
  "stage": "layout",  
  "status_url": "/jobs/20260727_110700_f071ed37",  
  "result_url": "/jobs/20260727_110700_f071ed37/result"  
}
```

6.3 내부 실행 순서

```
routes.py  
-> /jobs/layout/upload 요청 수신  
-> multipart form 파싱  
-> config JSON 파싱  
-> stage="layout" 설정  
-> PDF를 data/uploads/에 저장  
-> jobs.start_background_job() 호출  
-> job_id 반환  
  
jobs.py background thread  
-> status를 running으로 변경  
-> payload를 argparse Namespace로 변환  
-> 업로드 PDF를 input_file에 연결  
-> full_toc_core.run_pipeline_locked() 실행  
  
full_toc_core.py  
-> PDF 파싱  
-> data/parsed/*_headings_parsed.txt 저장  
-> OpenAI layout prompt 생성  
-> OpenAI API 호출  
-> layout JSON 정규화  
-> data/layout/*_layout.json 저장  
-> status completed 갱신
```

6.4 생성 파일

```
data/uploads/파일.pdf  
data/parsed/*_headings_parsed.txt  
data/parsed/*_parsed.txt  
data/layout/*_layout.json  
data/log/full_toc_*.log  
data/jobs/JOB_ID/status.json
```

7. 전체 TOC 생성 프로세스

7.1 요청

```
curl -X POST <http://127.0.0.1:8080/jobs/toc/upload> \
-F 'file=@../Contents_ai_test/data/input/1_2.pdf' \
-F 'config={"parse_mode":"headings"}'
```

7.2 내부 실행 순서

PDF 업로드
-> PDF parsed text 생성
-> OpenAI layout JSON 생성
-> layout JSON + parsed text로 Gemma prompt 구성
-> page/token 기준 chunk 분할
-> Gemma/vLLM API 호출
-> Gemma 응답 JSON 파싱
-> parsed layout line과 chapter title 매칭
-> page/source_order metadata 보정
-> 최종 TOC JSON 저장

7.3 생성 파일

```
data/layout/*_layout.json
data/full_json/*_full_foc.json
data/log/full_toc*.log
data/jobs/JOB_ID/status.json
```

8. Gemma만 실행 프로세스

8.1 요청

```
curl -X POST <http://127.0.0.1:8080/jobs/gemma/upload> \
-F 'file=@../Contents_ai_test/data/input/1_2.pdf' \
-F 'config={
  "parse_mode":"headings",
  "layout_file":"data/layout/0727_110700_1_2_0727_110700_
```



```
layout.json"
}'
```

8.2 내부 실행 순서

```
PDF 업로드
-> parsed text 확인
-> 없으면 PDF 파싱
-> 지정한 layout_file 로드
-> Gemma prompt 생성
-> Gemma/vLLM API 호출
-> 최종 TOC JSON 저장
```

9. parse_mode=headings 필터링

`parse_mode="headings"` 는 PDF 전체 본문을 그대로 모델에 넘기지 않고 heading 후보만 남기는 모드입니다.

9.1 유지 조건

- 제목 패턴에 맞는 줄
- 본문보다 큰 글자 크기
- bold이고 짧은 줄
- 번호 체계가 있는 heading 후보

9.2 제거 조건

- 본문 기본 크기와 같은 줄
- 너무 긴 본문형 문장
- 저자명 형태
- 작은 동그라미 번호 리스트
- heading 후보가 없는 페이지의 전체 본문 fallback

9.3 관련 코드

- `CIRCLED_NUMBER_RE`
- `should_keep_formatted_line()`

- `is_heading_like()`
- `extract_region_lines()`

이 필터는 본문 문장이 layout rule로 들어가는 문제를 줄이기 위한 것입니다.

기존에 생성된 layout JSON은 자동으로 바뀌지 않습니다. 필터를 수정한 뒤에는 서버를 재 시작하고 layout을 다시 생성해야 합니다.

10. Job 상태 확인

10.1 상태 조회

```
curl <http://127.0.0.1:8080/jobs/JOB_ID>
```

상태 값:

- `queued`
 - job 생성됨
- `running`
 - background thread에서 실행 중
- `completed`
 - 성공
- `failed`
 - 실패

10.2 결과 메타데이터 확인

```
curl <http://127.0.0.1:8080/jobs/JOB_ID/result>
```

10.3 실제 결과 파일 확인

```
curl <http://127.0.0.1:8080/jobs/JOB_ID/files/result>
```

파일로 저장:

```
curl -o result.json <http://127.0.0.1:8080/jobs/JOB_ID/files/result>
python -m json.tool result.json | less
```

`less` 에서 나가려면 `q` 를 누릅니다.

10.4 로그 확인

```
curl <http://127.0.0.1:8080/jobs/JOB_ID/files/log>
```

11. Upload 파일 경로 규칙

`curl -F 'file=@...'` 의 파일 경로는 FastAPI 서버 기준이 아닙니다.

`curl` 명령을 실행하는 터미널의 현재 위치 기준입니다.

업로드 전에 같은 경로를 `ls -lh` 로 확인합니다.

```
ls -lh ../Contents_ai_test/data/input/1_2.pdf
```

이 명령이 실패하면 `curl -F 'file=@...'` 도 실패합니다.

12. Smoke Test

12.1 실행

```
python smoke_test.py
```

12.2 확인하는 것

- FastAPI app import
- `/health`
- 없는 job 404
- 잘못된 upload 400
- PDF 업로드
- job 생성

- PDF 파싱
- layout/result/log 파일 생성
- 결과 파일 다운로드

12.3 확인하지 않는 것

- 실제 OpenAI API 연결
- 실제 Gemma/vLLM 서버 연결
- 모델 출력 품질
- 대용량 PDF 처리 성능

13. 자주 발생하는 오류

13.1 OpenAI API key 없음

오류:

```
OpenAI API key is empty. Set OPENAI_API_KEY or pass --openai-api-key.
```

해결:

- 프로젝트 루트의 `.env` 에 `OPENAI_API_KEY=sk-...` 가 있는지 확인합니다.
- `.env` 를 새로 만들거나 수정했다면 FastAPI 서버를 재시작합니다.

13.2 파일 업로드 경로 오류

오류:

```
curl: (26) Failed to open/read local data from file/application
```

원인:

- `file=@...` 경로가 존재하지 않음
- 상대경로를 서버 기준으로 착각함
- 한글 파일명이 깨져서 다른 이름으로 입력됨

해결:

```
ls -lh ../Contents_ai_test/data/input/1_2.pdf
```

13.3 Gemma/vLLM 서버 연결 실패

오류:

```
Gemma server is not reachable
```

확인:

```
curl <http://127.0.0.1:8000/v1/models>
```

다른 주소를 쓰는 경우 config에 `base_url` 을 지정합니다.

```
{
  "parse_mode": "headings",
  "base_url": "<http://127.0.0.1:8000/v1>"
}
```

13.4 이전 job_id 조회

새로 업로드했는데 이전 job_id를 조회하면 이전 실패 상태가 그대로 보입니다.

업로드 응답에서 받은 최신 `job_id` 를 사용해야 합니다.